

Microcontrollers and Real-Time Constraints

Course notes - Department of Electrical Engineering

1. Microcontroller architecture

A microcontroller integrates processor, memory and peripherals on one device. Harvard architectures separate instruction and data memory, allowing simultaneous access, whereas von Neumann designs share one bus.

Flash holds the program, SRAM holds working data, and memory-mapped registers expose peripherals. Programming embedded systems is largely the discipline of writing correct values to the right registers in the right order.

2. Interrupts

An interrupt suspends normal execution to run a handler, letting the processor respond to events without polling. The vector table maps each source to its handler, and priority determines what may preempt what.

Handlers must be short and must not block. Data shared with the main program requires either atomic access or explicit critical sections, and any variable modified in a handler must be declared volatile.

3. Timers and PWM

A timer counts clock ticks and can generate periodic interrupts, capture the time of an input edge, or drive an output. It is the basis of every scheduled activity on the device.

Pulse-width modulation varies duty cycle at fixed frequency to control average delivered power, which is how motor speed and LED brightness are set without analogue circuitry.

4. Communication interfaces

UART provides asynchronous point-to-point serial communication with both ends configured to the same baud rate. SPI is synchronous, full duplex and fast, using a chip select per peripheral.

I2C uses two wires shared by many devices, addressing each by a 7-bit address, trading throughput for pin count. The choice among them is usually dictated by the peripheral rather than by preference.

5. Real-time scheduling

A hard real-time system fails if a deadline is missed; a soft real-time system merely degrades. What matters is the worst-case execution time, not the average.

Rate-monotonic scheduling assigns static priority by frequency and is optimal among fixed-priority schemes. Earliest-deadline-first is dynamic and achieves full utilisation. Priority inversion, where a high-priority task waits on a resource held by a low-priority one, is addressed by priority inheritance.